PHISHING SIMULATION AND SECURITY AWARENESS PROGRAM

College Mini-Project Report

Submitted by: [Student Name]

Roll Number: [Roll Number]

Department / Class: [Department / Class]

Institution: [College / University]

Project Guide: [Guide Name]

Academic Year: [Academic Year]

Abstract and Problem Statement

This project shows a controlled phishing-awareness lab that uses GoPhish and MailHog. The goal was to learn how to create phishing emails, how to track when emails are opened and links are clicked and how a landing page can explain the warning signs of phishing. A handout and a short offline quiz were also made to aid the awareness part of the project.

The project focuses on a problem: people may react quickly to an email without checking who sent it or where a link goes. To study this safely every email test was routed to a MailHog inbox and used the dummy address student@example.test. No real people were. The landing page was set up not to collect passwords or any submitted data.

Scope: the work completed far demonstrates the technical setup and its behavior, in a local lab. The results do not prove that people became better at spotting phishing because a real training session and a controlled comparison of behavior have not yet been done.

1. Introduction and Background

I think email is used for communication and it is also used often in phishing attempts. These messages can look like password notices, delivery updates or urgent requests. They try to make the recipient act quickly before checking the sender or the link. Awareness training tells users to check the sender inspect links carefully avoid attachments and report suspicious messages.

I think of sending emails to real inboxes the project uses a local lab. GoPhish manages the templates and the campaign results while MailHog shows the messages sent by the test. A local landing page gives safety advice. Says it does not collect credentials. The quiz runs in a browser without a service and shuffles both the questions and the answer choices.

Project Objectives

- Configure GoPhish and MailHog for a only email simulation.
- Create two educational email scenarios: an IT password-expiry notice and a delivery notification.
- Record baseline and technical tracking-test metrics, from GoPhish.
- Create a phishing-awareness handout and a five-question randomized quiz.
- Demonstrate a landing page and verify that the test flow does not collect passwords or submitted data.
- Prepare a CSV-based campaign comparison and a Python updater that retrieves campaign data through the GoPhish API to final successful testing.

2. Lab Design and Tools

Component	Purpose	Configuration / Notes
GoPhish v0.12.1	Create email templates, campaigns, landing pages, and track events.	Local dashboard: https://127.0.0.1:3333
MailHog	Capture test email without sending it to a real external mailbox.	Web inbox: http://127.0.0.1:8025; SMTP: 127.0.0.1:1025
Local SMTP sending profile	Route GoPhish messages to MailHog.	Profile: MailHog Local Test; sender: lab@example.test
Dummy recipient group	Restrict campaign delivery to the lab recipient.	Local Lab Test Group; student@example.test
Awareness landing page	Display safe educational content after a tracking link is opened.	Local HTTP landing server on port 80; page states no credentials are collected.
HTML/CSS/JavaScript quiz	Reinforce phishing-recognition knowledge offline.	Five questions; randomized question and option order; score, explanations, reset.
Python + GoPhish API	Refresh campaign metrics into CSV.	Updater file created; final run should be verified after fixing the reported Notes-field error.

Lab Architecture

The workflow is simple: a campaign is created in GoPhish the message is sent through the SMTP profile to MailHog and the test message can be opened in the inbox. The tracking link opens the awareness page and GoPhish records available events. Campaign results can be viewed in GoPhish. The CSV updater is meant to copy these metrics into results/campaign-comparison.csv after its error is fixed.

The CSV updater is another automation feature. In the recent test it connected to the GoPhish API and fetched four campaigns but stopped before updating the CSV because a blank Notes value caused an error. This part is not yet done: the error must be. The script must run correctly before the automation can be considered working.

3. Methodology and Implementation (with Screenshots)

Environment preparation

We unpacked GoPhish on a Windows system. Launched it from the local project folder. I ran MailHog to catch test messages. I accessed the GoPhish dashboard and the MailHog inbox through localhost URLs.

Sending profile and dummy recipient

A GoPhish sending profile called "MailHog Local Test" was set up to use SMTP at 127.0.0.1:1025. The test group included the dummy email address student@example.test.

Email templates

Two templates were created: "IT Password Expiry Awareness Test" and "Delivery Notification Awareness Test." The delivery template was later updated to include a View the security awareness notice" link using the GoPhish https://127.0.0.1:3333 placeholder. The tracking-image option was enabled for a technical verification test.

Landing page safety

The landing page titled "Phishing Awareness Training Page" was set up with the options to Capture Submitted Data and Capture Passwords both turned off. The page provides guidance, on phishing awareness. Clearly states that it does not collect any login credentials.

Baseline and tracking tests

Campaign 1. Baseline Test went to the recipient. A different Campaign 1. Tracking Test was used to check if opens and clicks were being tracked. Campaign 2 was sent with the delivery notification template. Then Campaign 3. Tracking Verification was used to make sure the tracking link was visible and the landing page worked.

Awareness material and quiz

I prepared a handout in Markdown and HTML. I made a quiz with five multiple-choice questions. The browser tests that were reported covered validation, for questions, partial and perfect scoring, explanations, reset behavior and shuffling of question and answer order.

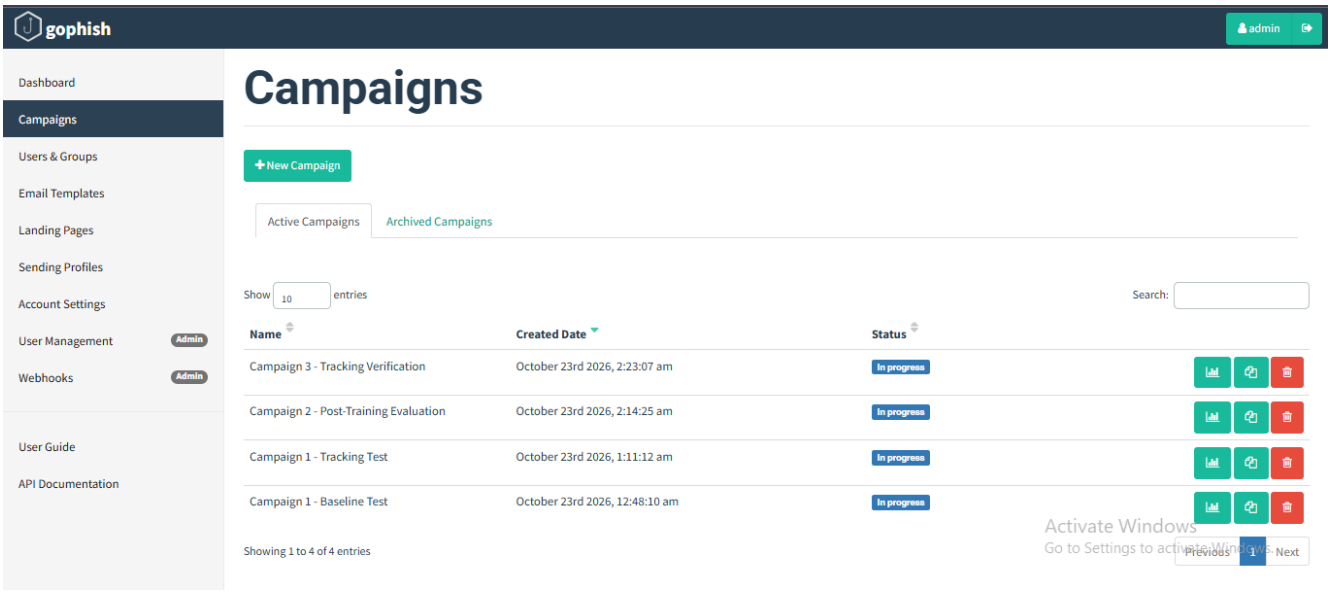
CSV updater

A Python updater was made to connect to the GoPhish API and update the file campaign-comparison.csv. It was able to connect to the API and get four campaigns.. A runtime error happened when it tried to handle a missing Notes value. The updater is still waiting for a fix and a full test run to finish successfully.

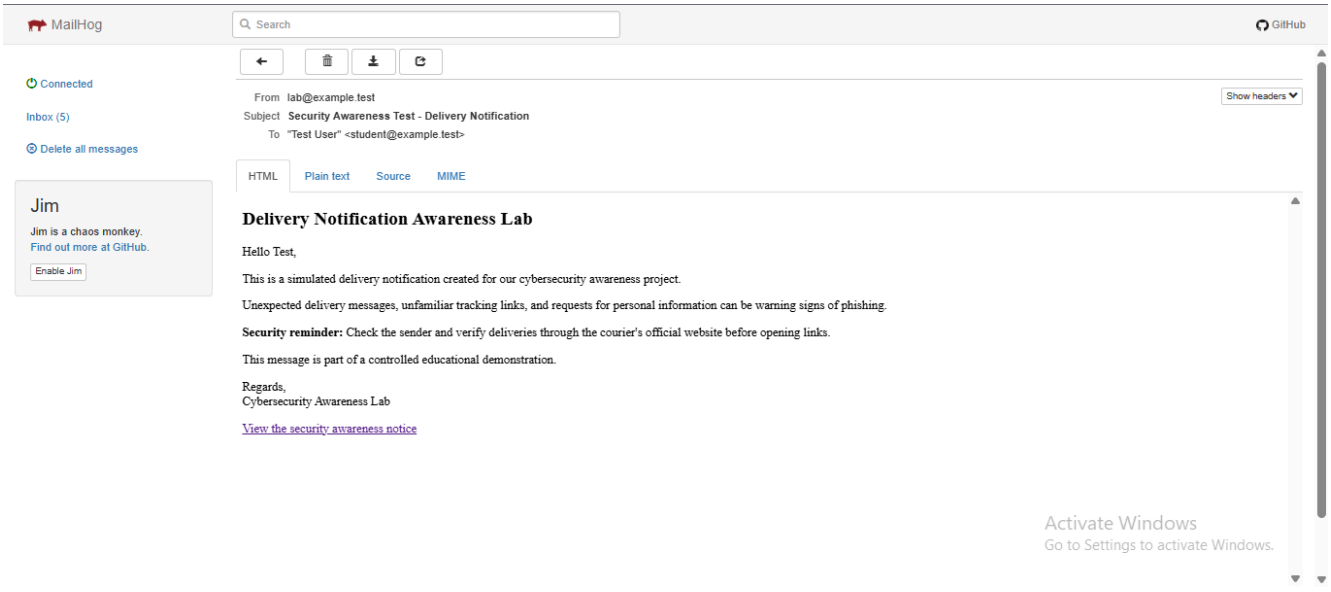
Screenshot Evidence Checklist

Insert the screenshots from the project screenshots/ folder at the places shown. Do not use placeholder images as evidence, in the submission.

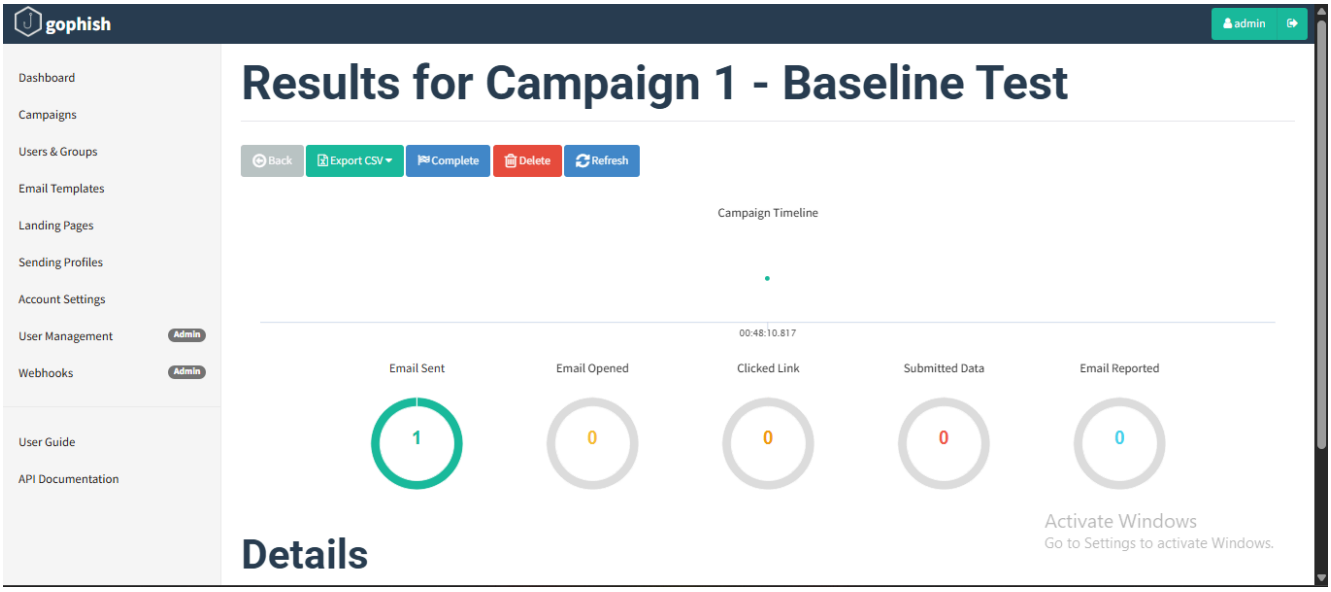
- Figure 1. GoPhish dashboard and campaign list.



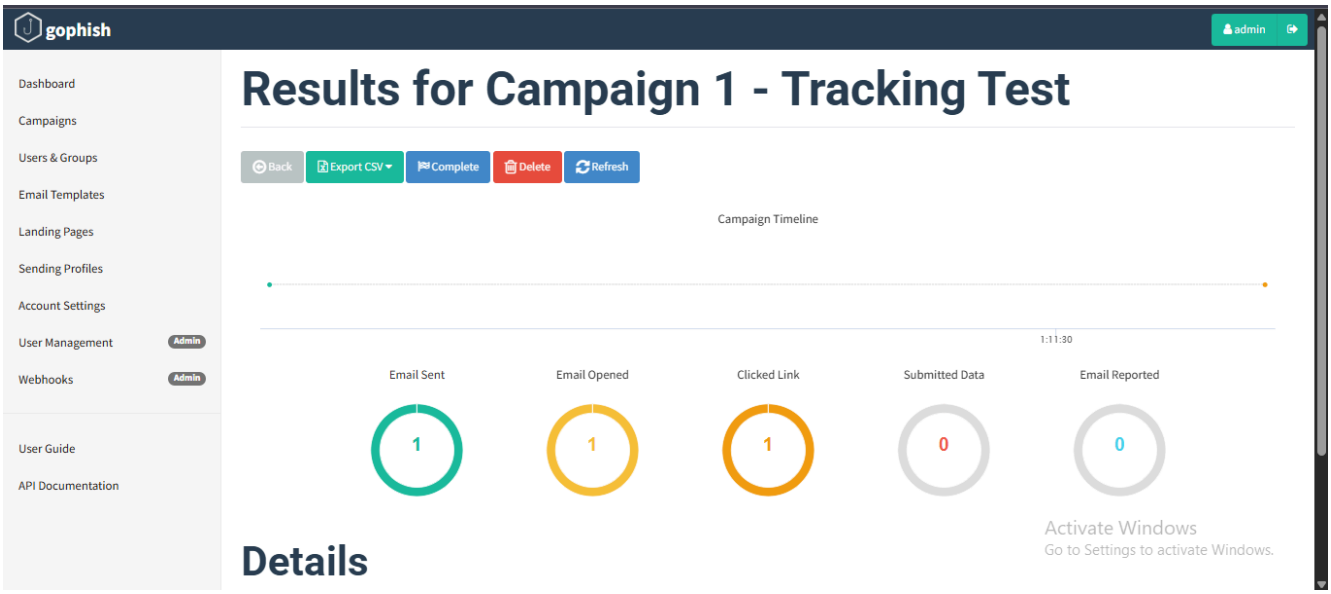
- Figure 2. MailHog inbox showing the test email.



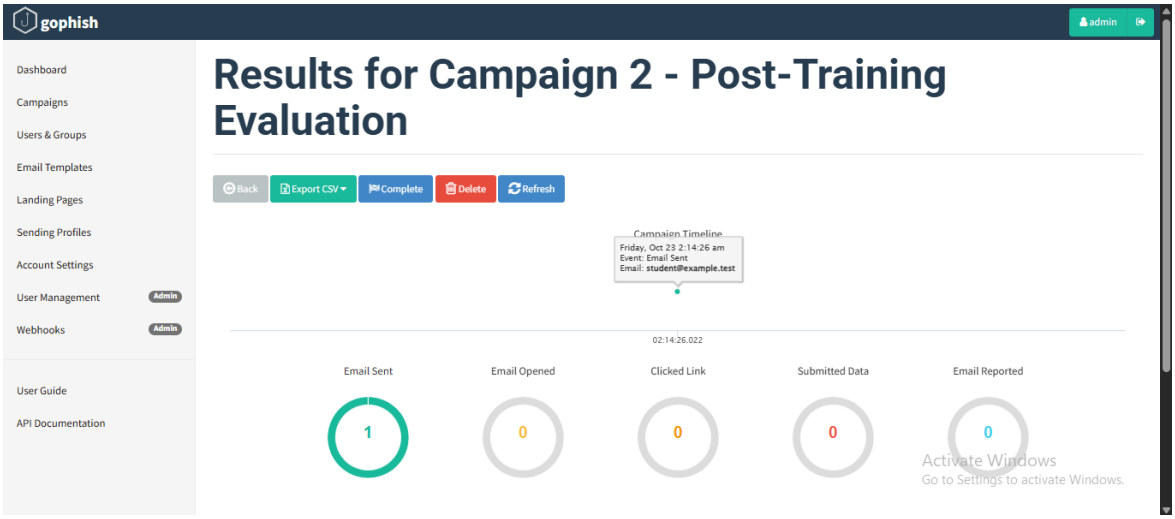
- Figure 3. Campaign 1. Baseline Test results.



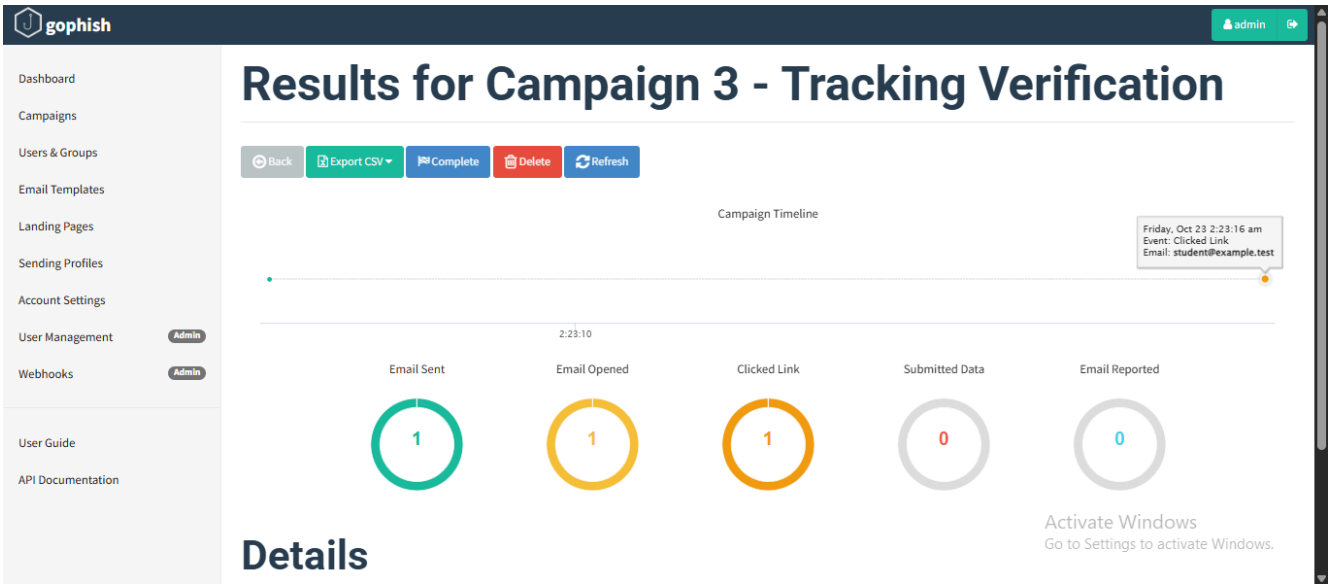
- Figure 4. Campaign 1. Tracking Test results.



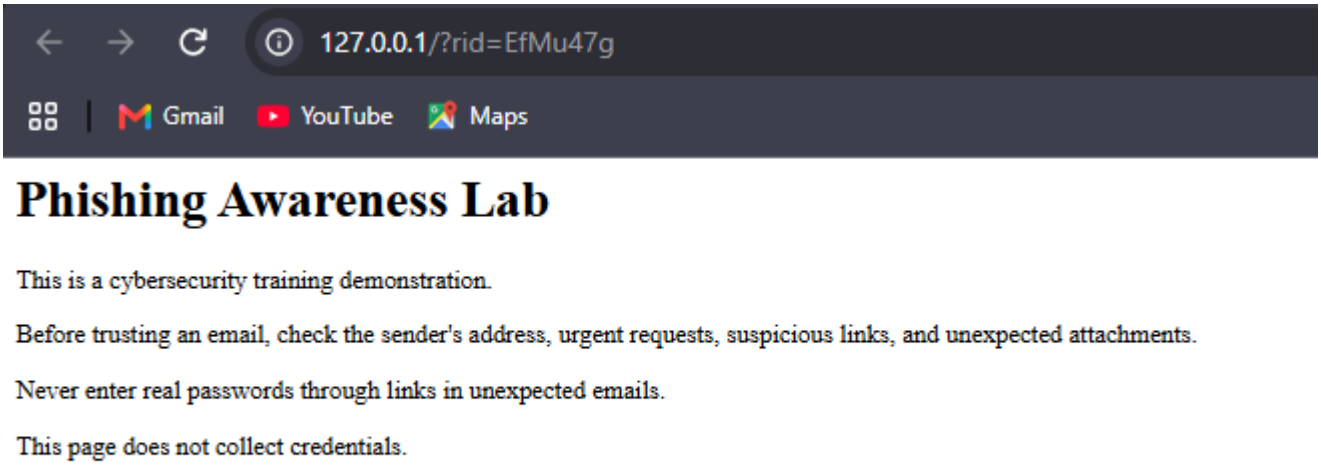
- Figure 5. Campaign 2. Post-Training Evaluation results.



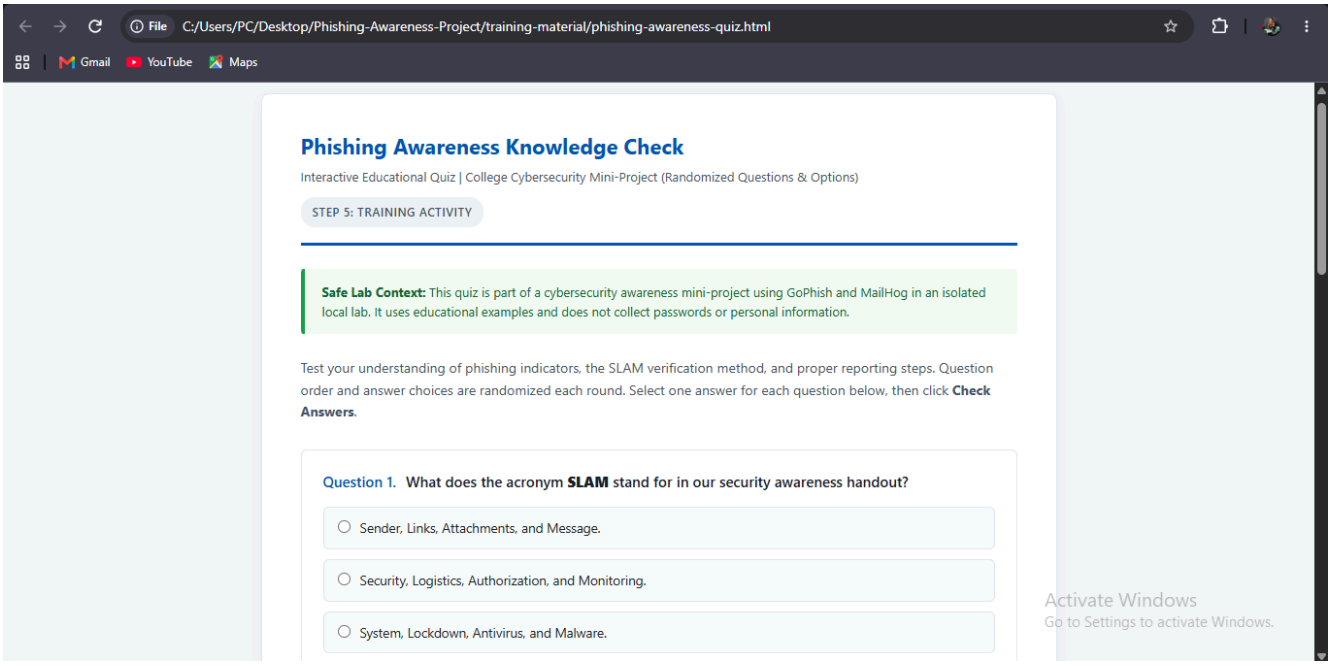
- Figure 6. Campaign 3. Tracking Verification results.



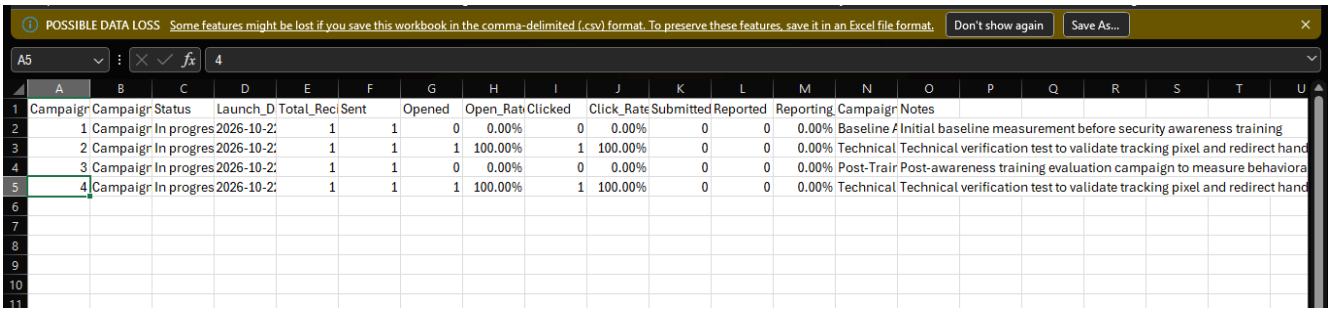
- Figure 7. Local phishing-awareness landing page.



- Figure 8. Randomized quiz showing score and explanations.



- Figure 9. Final campaign-comparison.csv opened in Excel or a text editor.



Campaign	Campaign	Status	Launch_D	Total_Rec	Sent	Opened	Open_Rat	Clicked	Click_Rate	Submitted	Reported	Reporting	Campaign	Notes
1	Campaign	In progres	2026-10-2	1	1	0	0.00%	0	0.00%	0	0	0.00%	Baseline A	Initial baseline measurement before security awareness training
2	Campaign	In progres	2026-10-2	1	1	1	100.00%	1	100.00%	0	0	0.00%	Technical	Technical verification test to validate tracking pixel and redirect hand
3	Campaign	In progres	2026-10-2	1	1	0	0.00%	0	0.00%	0	0	0.00%	Post-Train	Post-awareness training evaluation campaign to measure behaviora
4	Campaign	In progres	2026-10-2	1	1	1	100.00%	1	100.00%	0	0	0.00%	Technical	Technical verification test to validate tracking pixel and redirect hand

4. Results, Findings and Analysis

The table, below lists the counts that you can see in the GoPhish result screens during testing. These snapshots come from the lab not from a group of real participants. Refresh the campaign pages before you submit in case any counts have changed.

Campaign	Sent	Opened	Clicked	Submitted	Reported	Interpretation
Campaign 1 - Baseline Test	1	0	0	0	0	Original baseline observation.
Campaign 1 - Tracking Test	1	1	1	0	0	Technical tracking verification; not baseline evidence.
Campaign 2 - Post-Training Evaluation	1	0*	0*	0	0	Counts observed at the time of screenshot; refresh before final submission.
Campaign 3 - Tracking Verification	1	1	1	0	0	Link opened local awareness page; technical verification only.

Campaign 2 showed 1 sent 0 opened, 0 clicked 0 submitted and 0 reported at the time seen. Check the campaign results again before considering these as the numbers.

Findings

- The local email delivery path worked. Test emails appeared in MailHog. Test emails did not go to an external mailbox.
- GoPhish recorded one and one click in the separate tracking tests. The Campaign 3 tracking link opened the awareness landing page.
- The landing page presented security-awareness content. The landing page stated that it did not collect credentials. Capture Submitted Data was confirmed unchecked during setup. Capture Passwords was confirmed unchecked during setup.
- The quiz was reported as tested for validation. The quiz was reported as tested for scoring. The quiz was reported as tested for explanations. The quiz was reported as tested for reset. The quiz was reported as tested for question/option order.
- Campaign metrics from the tracking verification tests must remain separate from metrics. Campaign metrics from the tracking verification tests must remain separate, from post-training evaluation metrics.
- The current observations do not establish that training improved behavior. Participant training delivery must be documented. A comparable evaluation must be documented before making that conclusion.

5. Challenges Faced and Lessons Learned

Finding the correct local landing URL

Trying to open the root URL http://127.0.0.1 directly led to a 404 error. Later using a campaign tracking link opened the expected awareness page. This proved that the root URL alone was not a test of a campaign-specific tracking path.

Email template changes do not alter sent emails

A message that was already sent to MailHog did not get a link after the template was changed. A new controlled test was needed to make sure the updated template works correctly.

Tracking image and visible link

At first the template did not show a link in the email that was sent and the tracking-image option was off, in a later screenshot. When we turned on tracking and added a {.URL}} link, the technical check passed in Campaign 3.

API updater CSV error

The updater connected to GoPhish and retrieved four campaigns from GoPhish.

The updater crashed because the Notes value was None so we need handling of blank CSV fields and a successful rerun, for the campaigns.

API key handling

I discovered that the API key was accidentally exposed during setup and the API key must be regenerated. The API key should be entered into the PowerShell session as an environment variable and the API key must never be saved in source code or shared in screenshots.

Interpreting metrics

Open and click counts show that tracking events were recorded;. Open and click counts do not prove that a human read or understood the email. Local technical tests are not evidence of training effectiveness.

6. Conclusion and Future Work

The project successfully set up a phishing-awareness demonstration using GoPhish and MailHog. Two email templates were created the awareness landing page was. The randomized quiz was prepared. Campaign screens showed that local email delivery worked and that the tracking link opened the landing page during a verification test. The CSV updater has been started,.

It still needs a fix and a successful final run. Before submission the project still needs a check of the consent and lab-rules file, confirmation of whether any participants completed the training organized screenshots refreshed campaign counts, a successful CSV updater run and actual screenshots inserted into this document.

Future Work

- We must validate the API updater. The Notes-field handling must be fixed. If an API request fails we must keep the CSV file.
- You can optionally schedule the validated updater. Use the Windows Task Scheduler to do this.
- We will run a training session. We will ask for consent. We will record completion. We will not collect passwords or unnecessary personal data.
-
- If a pre-training and post-training study is needed we will create a consistent method. We will use approved participants. We will analyze results. We will not treat test campaigns as participant outcomes.
- We will extend awareness training. We will add scenarios. We will add a reporting procedure. We will add a post-training knowledge check.

7. References

- GoPhish. Official. Documentation. Https://getgophish.com/ (Accessed on April 5 2024 for project setup and campaign features; used to configure and run phishing simulations.)
- GoPhish API Documentation. Introduction, authorization, campaigns and reporting endpoints. Accessible from the GoPhish dashboard, via the API Documentation section.
- MailHog. Official project. Documentation. Https://github.com/mailhog/MailHog
- OWASP Foundation. Phishing guidance and security awareness resources. Https://owasp.org/ (Consulted the page titled "Phishing" on April 5 2024 for practices and mitigation strategies.)
- Project implementation artifacts: GoPhish campaign result screens, MailHog messages, training handout randomized quiz and campaign-comparison.csv. Include screenshots and files actually retained in the project.

Appendix A — Demonstration Commands

Start GoPhish (PowerShell window 1)

```
cd C:\Users\PC\Desktop\Phishing-Awareness-Project\GoPhish\gophish-v0.12.1-windows-64bit
.\gophish.exe
```

Start MailHog (PowerShell window 2)

```
cd C:\Users\PC\Desktop\Phishing-Awareness-Project\MailHog
Get-ChildItem *.exe
.\MailHog_windows_amd64.exe  # adjust to the exact filename shown
```

Open project root (PowerShell window 3)

```
cd C:\Users\PC\Desktop\Phishing-Awareness-Project
```

Run updater after fixing and testing it

```
$env:GOPHISH_API_KEY = Read-Host "Enter your GoPhish API key"
python scripts\update_campaign_results.py
```

View CSV

```
Get-Content .\results\campaign-comparison.csv
```

Open quiz

```
start .\training-material\phishing-awareness-quiz.html
```

Appendix B — Final Verification Checklist

- ☐ Replace all cover-page placeholders with the student and institution details.
- ☐ Verify that the consent and lab-rules document exists and matches the lab.
- ☐ Refresh GoPhish campaign. Update the results table if counts have changed.
- ☐ Fix the updaters Notes-field error. Confirm a successful CSV update before claiming automation works.
- ☐ Insert screenshots, into the report and ensure filenames match the project folder.
- ☐ Confirm whether training was delivered; do not claim it was delivered if only self-testing occurred.
- ☐ Review. Apply the citation style required by the institution.